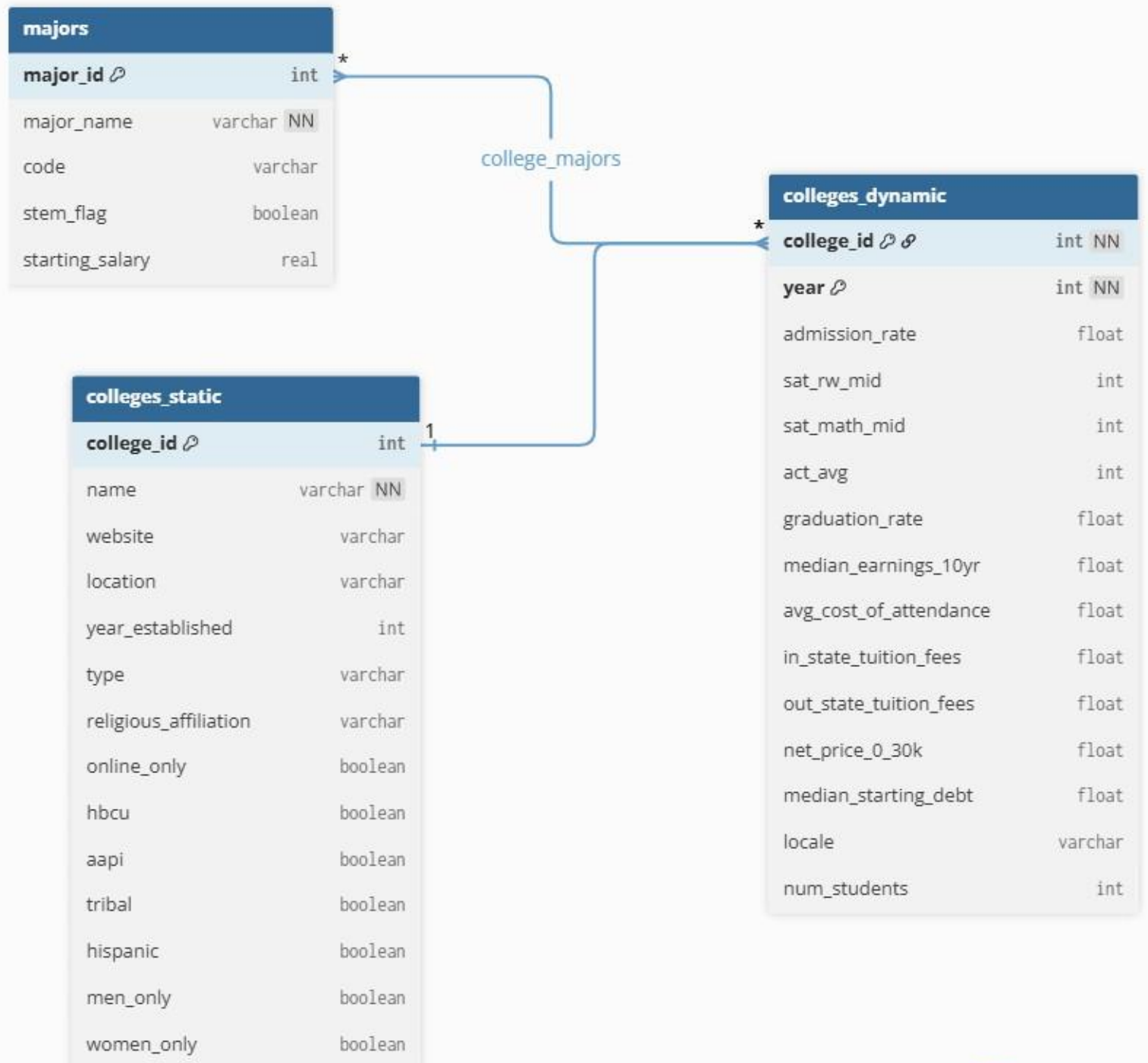
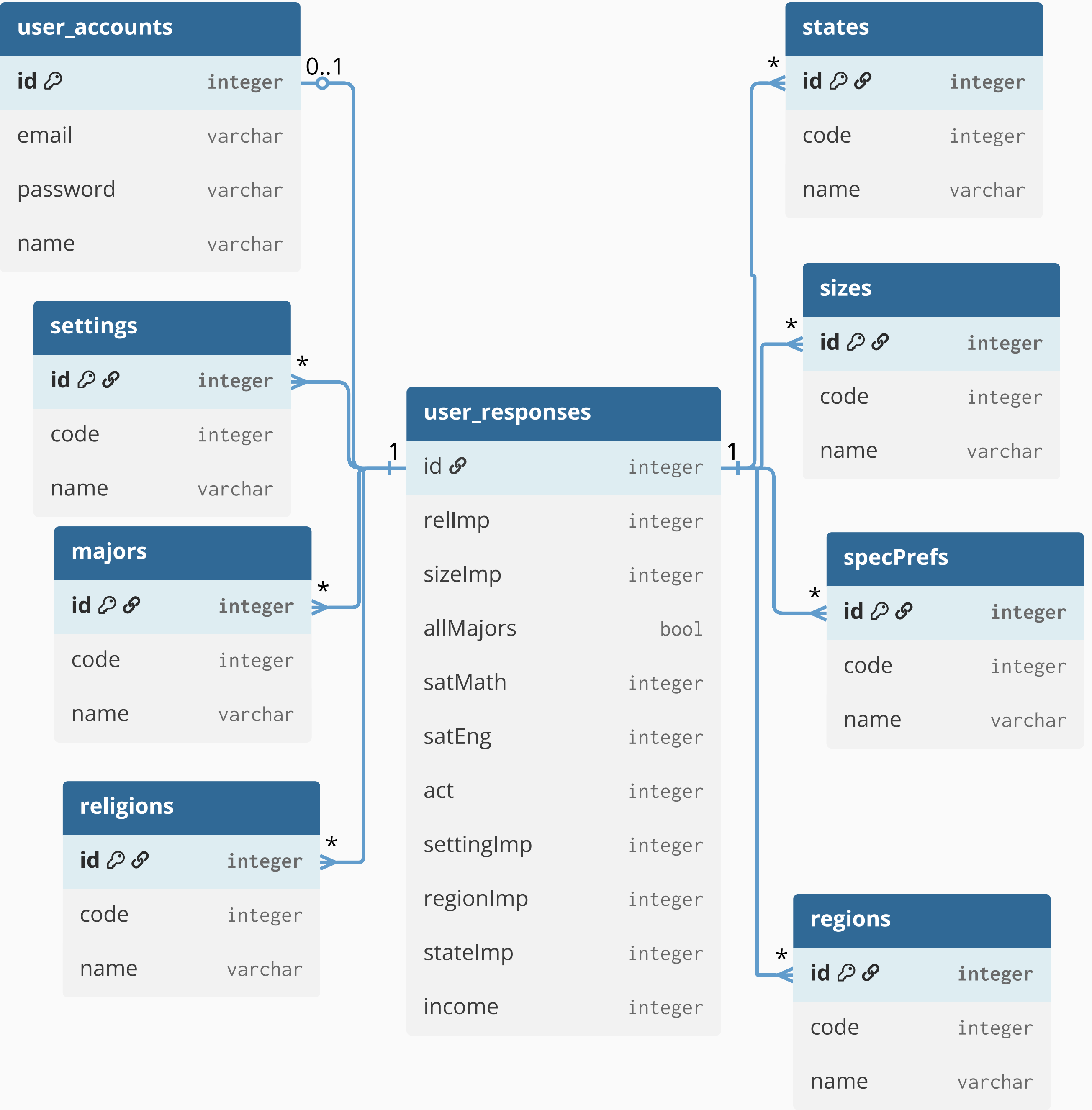






Assumptions and Explanations

User Side

- **user_responses** table: this is the entity set containing every users' responses to the form on the website, such as their SAT and ACT scores, the relative importance (on a scale of 1-10) of their college's religious affiliation, state, region, and setting (urban vs rural vs suburban/small town), their income bracket, and whether they only want to see school that have all of their selected majors. This is the primary table of the users' side of the database. Every website user will have a unique entry in this entity set with a unique primary key **id**, corresponding to the id of that user as assigned by the system. There is only 1 unique row per user, which is updated whenever a user submits a new form, so the cardinality of every relationship the **user_responses** table has is 1 on its side (i.e. every other table that references the **user_responses** can relate to only 1 entry in it)
- **user_accounts** table: this is the entity set containing the users' account information, including email, password, and name. There are two reasons why this is a separate table from **user_responses**: first, because it contains information that the user does not enter in the main form and is thus should have different security permissions/be accessed by different parts of the program, and second, most importantly, not every user will have an account. Our website offers the opportunity for users to use it anonymously, in which case an entry is created in **user_responses** table but not in **user_accounts**. If a user does sign in, an entry is created in the **user_accounts** table that references the same id in **user_responses**. Therefore, the relationship between **user_accounts** and **user_responses**

is a one-one relationship, with every entry in **user_account** having exactly 1 corresponding row in **user_response**, and every entry in **user_response** has at most 1 (0..1) entry in **user_account**.

- Supporting attributes tables: these are the user responses to questions that can have multiple answers. For example, a user can select multiple states that they are interested in going to college in. Therefore, to maintain atomicity of records, these could not be simply entered in the **user_responses** table. Each one has a many-one relationship with **user_responses** as a result. Every entry in each of these “supporting attributes” tables has to reference exactly one row in **user_responses** through their foreign key **user_id**, but every row in **user_responses** can be connected to any number of entries in these tables (0..*). For example, a user may have no preference for state at all, may be open to colleges in 1 state, or in several states. Then, that user’s entry in **user_responses** will be connected to 0 entries in the **states** table, to 1 entry, or to multiple entries, respectively. Each of these tables will have a numeric code for the entry and a string name (e.g. “computer science” in the **majors** table can have a code of 23). These *supporting attributes* tables are as follows:
 - **religions**: a user’s preferred college religious affiliation.
 - **sizes**: a user’s preferred college size (e.g. small (<5,000 students), medium (5,000-15,000), or large (>15,000)).
 - **majors**: fields of study a user is interested in
 - **settings**: a user’s preferred college setting (urban, suburban/small town, rural)
 - **regions**: a user’s preferred region for their college location (e.g. Midwest, New England, etc.)

- **states:** a user's preferred state for their college location, including US territories
- **specPrefs:** any special preferences of the user (e.g. HBCU college, minority-serving institution, gender-specific, or online-only).

College Side

- **colleges_dynamic table:**
 - **Assumption/meaning:**
 - colleges_dynamic stores a yearly snapshot of a college's changeable statistics (e.g., admission rate, number of students, graduation rate, cost/outcome metrics).
 - **Why this is an entity (not just attributes on colleges_static):**
 - These values are time-dependent. A college can have different statistics across years, so storing them in colleges_static would force overwriting previous values and would lose historical data. By modeling "college-year statistics" as its own entity, the database preserves history and supports queries like trends over time and comparisons across years.
 - **Uniqueness / keys assumption:**
 - Each record is uniquely identified by (college_id, year), enforcing the constraint that there is at most **one row per college per year**.
 - **Relationship + cardinality:** colleges_static (1) → (0...*) colleges_dynamic
 - One college may have zero or many yearly statistic rows (e.g., missing years or incomplete data).
 - Each colleges_dynamic row belongs to exactly one college.

- **colleges_static table**

- **Assumption / meaning:**

- colleges_static represents a single college as a stable real-world institution.

- It contains attributes that are mostly stable over time (e.g., name, location, website, year established).

- **Why this is an entity:**

- Multiple parts of the database (yearly stats, majors offered, and future user-facing features like favorites or recommendations) must reference a college. That requires a standalone entity with a primary key (college_id).

- **Stability assumption:**

- college_id is the true identifier and is assumed to never change. The name is treated as unique in the dataset, but it is not used as the primary key because names can change and are less reliable than surrogate IDs.

- **Relationship implications:**

- This table anchors other relationships so identity data is stored once rather than duplicated across yearly or relationship tables.

- **majors table**

- **Assumption / meaning:**

- majors represents a major/field of study as a reusable concept across many colleges (e.g., Computer Science, Nursing). It includes a unique major_id, a code (if available), a stem_flag for classification/filtering, and starting_salary as an approximate baseline outcome associated with the major itself.

- **Why this is an entity (not an attribute of college):**
 - Majors are not specific to one college; they are shared across institutions.
Storing majors inside each college record would create duplication and inconsistency (e.g., “CS” defined slightly differently across rows). Also, the relationship between colleges and majors is naturally many-to-many, which requires modeling majors independently.
- **Cardinality:**
 - Every college can offer several majors, and every major can be offered by several colleges
 - Therefore, the **majors** table has a many-to-many relationship with the **colleges_dynamic** table
 - colleges_dynamic (0..*) → (0..*) majors

Normalization (BCNF)

User Side

user_accounts: id is the primary key, email is an unique attribute, both id and email are superkey, thus every non-trivial functional dependency has a superkey as its determinant, so user_accounts is in BCNF.

user_responses: id is the primary key, all other attributes are dependent on id, there's no other functional dependencies in this relation, so user_responses is in BCNF.

However, for the preference tables (religions, majors, states, etc.), each table has a functional dependency code $\rightarrow$ name. Code is not a key, so these relationships, as they're written, violate CBTF. To normalize these tables, they each need to be split into a mapping table with code as a primary key and name as an attribute, and a response table with user_id and codes, with a composite primary key. The normalized schema is as follows:

```
religions_codes(  
    code: INT [PK],  
    name: VARCHAR (255)  
)  
  
user_religions(  
    user_id [FK  $\rightarrow$  user_responses.id],  
    code [FK  $\rightarrow$  religions_codes.code],  
    PRIMARY KEY (user_id, code)  
)  
  
sizes_codes(  
    code: INT [PK]
```

```

        name: VARCHAR (255)
    )

    user_sizes (
        user_id: [FK→ user_responses.id]
        code [FK → sizes_codes.code]
        PRIMARY KEY (user_id, code)
    )

    majors_codes(
        code: INT [PK],
        name: VARCHAR (255)
    )

    user_majors(
        user_id [FK → user_responses.id],
        code [FK → majors_codes.code],
        PRIMARY KEY (user_id, code)
    )

    settings_codes(
        code: INT [PK]
        name: VARCHAR (255)
    )

    user_settings (
        user_id: [FK→ user_responses.id]
        code [FK → settings_codes.code]
        PRIMARY KEY (user_id, code)
    )

    regions_codes(
        code: INT [PK],

```

```

        name: VARCHAR (255)
    )

    user_regions(
        user_id [FK → user_responses.id],
        code [FK → regions_codes.code],
        PRIMARY KEY (user_id, code)
    )

    states_codes(
        code: INT [PK]
        name: VARCHAR (255)
    )

    user_states(
        user_id: [FK→ user_responses.id]
        code [FK → states_codes.code]
        PRIMARY KEY (user_id, code)
    )

    specPrefs_codes(
        code: INT [PK],
        name: VARCHAR (255)
    )

    user_specPrefs(
        user_id [FK → user_responses.id],
        code [FK → specPrefs_codes.code],
        PRIMARY KEY (user_id, code)
    )

```

College Side

colleges_static: college_id is the Primary Key, all the non-key attributes are dependent on college_id, there's no other functional dependencies, so colleges_static is in BCNF.

colleges_dynamic: the composite key (college_id, year) can identify each yearly record, and other attributes are fully dependent on this composite key, so colleges_dynamic is in BCNF.

majors: major_id is the Primary Key, all other attributes are dependent on major_id. There's no additional functional dependency, so majors is in BCNF.

college_majors: there are no non-key attributes in this relation, so it's in BCNF.

Logical Design and Relational Schema

User Side

user_responses(

id: INT [PK],

relImp: INT,

sizeImp: INT,

allMajors: BOOLEAN,

satMath: INT,

satEng: INT,

act: INT,

settingImp: INT,

regionImp: INT,

stateImp: INT,

income: INT

)

user_accounts(

id: INT [PK, FK to user_responses.id],

email: VARCHAR(255),

password: VARCHAR(255),

name: VARCHAR(255)

)

religions_codes(

code: INT [PK],

name: VARCHAR (255)

)

user_religions(

user_id [FK to user_responses.id],

code [FK to religions_codes.code],

PRIMARY KEY (user_id, code)

)

sizes_codes(

code: INT [PK]

name: VARCHAR (255)

)

user_sizes (

user_id: [FK to user_responses.id]

code [FK to sizes_codes.code]

PRIMARY KEY (user_id, code)

)

majors_codes(

```
        code: INT [PK],
        name: VARCHAR (255)
    )
user_majors(
    user_id [FK to user_responses.id],
    code [FK to majors_codes.code],
    PRIMARY KEY (user_id, code)
)
settings_codes(
    code: INT [PK]
    name: VARCHAR (255)
)
user_settings (
    user_id: [FK to user_responses.id]
    code [FK to settings_codes.code]
    PRIMARY KEY (user_id, code)
)
regions_codes(
    code: INT [PK],
    name: VARCHAR (255)
)
user_regions(
    user_id [FK to user_responses.id],
    code [FK to regions_codes.code],
    PRIMARY KEY (user_id, code)
)
```

```
states_codes(  
    code: INT [PK]  
    name: VARCHAR (255)  
)  
  
user_states(  
    user_id: [FK to user_responses.id]  
    code [FK to states_codes.code]  
    PRIMARY KEY (user_id, code)  
)  
  
specPrefs_codes(  
    code: INT [PK],  
    name: VARCHAR (255)  
)  
  
user_specPrefs(  
    user_id [FK to user_responses.id],  
    code [FK to specPrefs_codes.code],  
    PRIMARY KEY (user_id, code)  
)
```

College Side

```
colleges_static(  
    college_id: INT [PK],  
    name: VARCHAR(255),  
    website: VARCHAR(255),  
    location: VARCHAR(255),
```

```

    year_established: INT,
    type: VARCHAR(255),
    religious_affiliation: VARCHAR(255),
    online_only: BOOLEAN,
    hbcu: BOOLEAN,
    aapi: BOOLEAN,
    tribal: BOOLEAN,
    hispanic: BOOLEAN,
    men_only: BOOLEAN,
    women_only: BOOLEAN
)

colleges_dynamic(
    college_id: INT [PK, FK to colleges_static.college_id],
    year: INT [PK],
    admission_rate: FLOAT,
    sat_rw_mid: INT,
    sat_math_mid: INT,
    act_avg: INT,
    graduation_rate: FLOAT,
    median_earnings_10yr: FLOAT,
    avg_cost_of_attendance: FLOAT,
    in_state_tuition_fees: FLOAT,
    out_state_tuition_fees: FLOAT,
    net_price_0_30k: FLOAT,
    median_starting_debt: FLOAT,
    locale: VARCHAR(255),

```

```
        num_students: INT
    )
```

(Composite Primary Key: college_id + year)

```
majors(
    major_id: INT [PK],
    major_name: VARCHAR(255),
    code: VARCHAR(50),
    stem_flag: BOOLEAN,
    starting_salary: FLOAT
)
```

```
college_majors(
    college_id: INT [PK, FK to colleges_dynamic.college_id],
    major_id: INT [PK, FK to majors.major_id]
    PRIMARY KEY (college_id, major_id)
)
```